

缓存

一、缓存雪崩

缓存雪崩指的是在短时间内大量缓存失效，导致大量请求直接打到数据库上，从而造成数据库压力过大，甚至崩溃的情况。

1 发生情况

- 1) **缓存集中失效：**大量缓存数据设置了相同或接近的过期时间，导致这些缓存数据在同一时间或短时间内失效。
- 2) **缓存服务器宕机：**缓存服务器发生故障或宕机，导致缓存数据不可用。
- 3) **网络故障：**网络故障导致缓存服务器无法被访问。

2 表现

- 1) **瞬时数据库压力剧增：**大量请求直接打到数据库，数据库负载急剧上升。
- 2) **系统响应时间变长：**由于数据库压力大，查询速度变慢，导致系统响应时间增加。
- 3) **可能引发数据库崩溃：**如果数据库无法承受高负载，可能会崩溃或无法响应请求，导致系统不可用。
- 4) **服务不可用：**系统可能会出现大量请求超时、服务不可用等情况。

3 预防措施

- 1) **缓存过期时间分散：**设置不同的缓存过期时间，避免集中失效。
- 2) **多级缓存架构：**使用本地缓存、分布式缓存和数据库多级缓存架构，分散压力。
- 3) **熔断机制和限流：**在缓存失效的情况下，使用熔断机制和限流策略，保护数据库不被压垮。

二、缓存击穿

缓存击穿是指在高并发的情况下，某个热点数据在缓存过期后，大量请求瞬间直接打到数据库，造成数据库压力骤增。

1 发生情况

- 1) **热点数据失效：**高并发情况下，某个热点数据在缓存过期后，大量请求同时访问该数据，直接打到数据库。
- 2) **缓存失效未及时重建：**缓存失效后，新的缓存数据未能及时建立。

2 表现

- 1) **瞬时数据库压力剧增：**大量请求同时访问数据库中的热点数据，数据库负载急剧上升。
- 2) **系统响应时间变长：**由于热点数据查询请求大量打到数据库，导致响应时间变长。
- 3) **可能引发短暂的服务不可用：**在高并发情况下，数据库负载过大

可能导致服务短暂不可用或请求超时。

3 预防措施

- 1) **热点数据永不过期**：对特别重要的热点数据设置为永不过期，手动管理缓存更新。
- 2) **使用互斥锁 (Mutex)**：在缓存失效时，使用互斥锁控制只有一个请求能访问数据库并重建缓存，其他请求等待缓存重建完成。
- 3) **缓存预热**：在缓存过期之前提前重建缓存，确保缓存数据始终有效。

三、Redis 分布式缓存

1 安装和配置 Redis

- 1) 安装 redis:

```
sudo apt-get update
sudo apt-get install redis-server
```

- 2) 配置 Redis 集群:

修改 ‘redis.conf’ 文件，启用集群模式:

```
cluster-enabled yes
cluster-config-file nodes.conf
cluster-node-timeout 5000
appendonly yes
```

3) 启动 Redis 实例:

```
redis-server /path/to/redis.conf
```

4) 创建 Redis 集群:

使用 redis-cli 工具创建集群:

```
redis-cli --cluster create <node1>:6379 <node2>:6379 <node3>:6379 --cluster-replicas
```

2 客户端连接

使用 Jedis 客户端连接 Redis 集群:

```
Set<HostAndPort> nodes = new HashSet<>();
nodes.add(new HostAndPort("127.0.0.1", 6379));
nodes.add(new HostAndPort("127.0.0.1", 6380));
nodes.add(new HostAndPort("127.0.0.1", 6381));

JedisCluster jedisCluster = new JedisCluster(nodes);

// 操作缓存
jedisCluster.set("key", "value");
String value = jedisCluster.get("key");
```

3 监控和管理

1) 使用 Redis Sentinel 监控和管理:

配置 Sentinel:

```
sentinel monitor mymaster 127.0.0.1 6379 2
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
sentinel parallel-syncs mymaster 1
```

2) 启动 Sentinel:

```
redis-sentinel /path/to/sentinel.conf
```

四、3 台 linux 实现

1 环境准备

假设有 3 台 Linux 服务器:

Server A: 192.168.1.1

Server B: 192.168.1.2

Server C: 192.168.1.3

2 安装 Redis

在每台服务器上安装 Redis:

```
sudo apt-get update
```

```
sudo apt-get install redis-server
```

3 配置 Redis

在每台服务器上配置 Redis，使其支持集群模式。假设在每台服务器上运行两个 Redis 实例，一个主节点，一个从节点。

复制 Redis 配置文件:

```
sudo cp /etc/redis/redis.conf /etc/redis/redis-7000.conf  
sudo cp /etc/redis/redis.conf /etc/redis/redis-7001.conf
```

编辑配置文件：

修改 `/etc/redis/redis-7000.conf` 和 `/etc/redis/redis-7001.conf` 文件中的以下内容：

```
port 7000 # 7001 for the second instance
cluster-enabled yes
cluster-config-file nodes-7000.conf # nodes-7001.conf for the second instance
cluster-node-timeout 5000
appendonly yes
```

启动 Redis 实例：

```
sudo redis-server /etc/redis/redis-7000.conf
sudo redis-server /etc/redis/redis-7001.conf
```

4 创建 Redis 集群

在任意一台服务器上使用 `redis-cli` 工具创建 Redis 集群。

连接到任意一台服务器：

```
ssh user@192.168.1.1
```

创建集群：

```
redis-cli --cluster create 192.168.1.1:7000 192.168.1.1:7001
192.168.1.2:7000 192.168.1.2:7001 192.168.1.3:7000 192.168.1.3:7001
--cluster-replicas 1
```

5 验证集群

使用 `redis-cli` 连接到集群，验证集群状态。

连接到 Redis 集群：

```
redis-cli -c -h 192.168.1.1 -p 7000
```

检查集群信息：

```
cluster info  
cluster nodes
```

五、实现理由

1. 高可用性：

多节点配置：在每台服务器上运行两个实例，一个作为主节点，一个作为从节点，确保每个数据都有备份，提高数据可靠性和可用性。

分布式架构：通过在不同服务器上部署 Redis 实例，避免单点故障，提高系统的高可用性。

2. 负载均衡：

一致性哈希：通过一致性哈希算法，将数据均匀分布到不同节点上，确保负载均衡。

主从复制：在发生故障时，自动切换到备份节点，保持系统稳定运行。

3. 性能优化：

分布式缓存：缓存数据分布在多个节点上，能够处理更大的数据量和更高的并发请求。

内存存储：Redis 采用内存存储，数据访问速度极快，显著提高系统性能。

4. 灵活性和扩展性:

集群扩展: Redis 集群可以方便地添加或移除节点, 灵活扩展缓存容量和处理能力。

配置简单: Redis 的配置和部署相对简单, 易于管理和维护。